

Adjoint-Based Aerodynamic Shape Optimization for the Discontinuous Galerkin Approach

Manmeet Bhabra

Bachelor of Engineering

Department of Mechanical Engineering

McGill University

Montreal, Quebec

December 2017

A thesis submitted to McGill University in partial fulfilment of the requirements of
the Undergraduate Honours Program

© Manmeet Bhabra, Siva Nadarajah

DEDICATION

To My Parents.

ACKNOWLEDGEMENTS

I would like to first and foremost express my gratitude to my supervisor and mentor, Professor Siva Nadarajah. His continued support and guidance has made my experience at the McGill Computational Aerodynamics Group a very memorable and enriching experience. I am truly grateful for having had the opportunity to work with him throughout my undergraduate studies.

I would like to also thank the members of the McGill Computational Aerodynamics Group, who have all been very helpful and supportive. I am extremely grateful to Philip Zwanenburg with whom I had the chance to work on several portions of this project with. His patience and help was instrumental in obtaining many of the results in this thesis. A very special thanks also to Nicolas Ringue and Doug Shi-Dong, who were always more than willing to offer their help and support whenever I had any questions.

Finally, I would like to thank my parents and sister. Their continued love and support has truly made me the person that I am today.

ABSTRACT

High-order methods have been the focus of much research in the past decade due to its higher order of accuracy relative to current CFD approaches in place. This work studies the use of high-order methods, more specifically the Discontinuous Galerkin method, paired with adjoint-based optimization in aerodynamic shape optimization applications. Simple optimization cases are first studied to check the capabilities of high-order approaches when used in optimization settings. Moreover, the benefits of higher orders of accuracy in adjoint-based optimization is also studied. In particular, the CPU time needed as well as the number of design iterations needed until convergence to the optimal solution is studied to see if the increased accuracy resulting from these methods proves to offer advantages in optimization applications.

ABRÉGÉ

Dans les dernières années les méthodes d'ordre supérieur ont été le centre de plusieurs recherches à cause de leur meilleure précision comparée à les approches traditionnels CFD. Ce document académique étudie la méthode de Galerkin Discontinue, une fonction d'ordre supérieur couplé avec l'optimisation à base d'adjoint, dans l'intérêt d'optimiser une forme aérodynamique. Des cas de simple optimisation sont étudiés en premier pour saisir la capacité des méthodes d'ordre supérieur. Ensuite, les avantages liés à la meilleure précision de l'optimisation à base d'adjoint sont aussi discutés. En particulier, le temps de processeur et le nombre d'itérations requis pour converger vers une solution optimale est observés dans le but de savoir si ces optimisations sont toujours avantageux comparé aux approches traditionnelles.

TABLE OF CONTENTS

DEDICATION	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
ABRÉGÉ	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
1 Introduction	1
1.1 Contribution	2
1.2 Thesis Overview	3
2 The Governing Equations	4
2.1 2D Euler Equations	4
2.2 The Discontinuous Galerkin Method	5
2.3 Bézier Functions as a Basis for Analysis	9
3 Optimization Framework	17
3.1 The Adjoint Approach	17
3.2 Radial Basis Function Mesh Deformation	20
3.3 Optimization Strategy	23
4 Validation Cases	27
4.1 Orders of Accuracy	27
4.1.1 The Gaussian Bump Case	27
4.1.2 Boundary Conditions	30
4.1.3 Results	30

4.2	Inverse Design Cost Function Convergence	32
5	Optimization Results	35
5.1	Inviscid Channel Inverse Design	35
5.2	NACA 0012 Inverse Design	40
5.3	NACA 0012 Target Lift Coefficient	44
6	Conclusion	50
6.1	Future Work	51
	REFERENCES	53

LIST OF TABLES

<u>Table</u>		<u>page</u>
4-1	Normalized entropy error convergence for $P = 1, 2, 3$ using superparametric elements with $P_G = P + 1$	31
4-2	Inverse design cost function convergence for $P = 1, 2, 3$ using superparametric elements with $P_G = P + 1$. The target mesh with a dimension of 96×16 was fixed for each P value.	33
5-1	The properties of the meshes used for the target lift coefficient optimization case.	45

LIST OF FIGURES

<u>Figure</u>		<u>page</u>
2-1	A schematic of the computational domain and the same domain when partitioned into elements.	6
2-2	Plots of the 1D Bézier basis functions for $P = 1$ (a), $P = 2$ (b) and $P = 3$ (c).	10
2-3	Plots of the 2D Bézier basis functions for $P = 2$	11
2-4	Plots of a sample Bézier Curve (a) and Bézier Patch (b) for $P = 3$. .	12
2-5	A schematic showing the mapping from the physical space to the common reference element.	14
3-1	RBF mesh deformation example	23
3-2	A summary of the optimization design cycle.	25
4-1	A plot of the $P_G = 3$, 48×8 Gaussian bump mesh	29
4-2	A plot of the normalized pressure distribution for the $P = 2$, 96×16 case	32
5-1	The initial and target mesh used for the optimization.	37
5-2	The initial, optimized and target profiles for the inviscid channel bump optimization case. The points shown are the design control points.	38
5-3	The optimized and target pressure distribution over the bump normalized by the total pressure. The x axis displays the arc length position along the channel normalized by its total arc length.	39
5-4	The convergence of the cost function and L2 norm of its gradient with respect to the design iteration for the inviscid channel case.	40

5-5	A plot of the NACA0012 and NACA4412 airfoil profiles.	41
5-6	The mesh placed around the NACA 0012 design airfoil that was optimized.	42
5-7	The initial, optimized and target profiles. The mesh dimension is 40 x 20 and $P = 2$	43
5-8	The convergence of the cost function and L2 norm of its gradient with respect to the design iteration for the NACA 0012 inverse design case.	44
5-9	Initial and optimized airfoil profiles for the target lift coefficient optimization case. Shown in brackets in the legend is the dimension the mesh used for the given P case.	47
5-10	Convergence of the lift coefficient as a function of design iterations and CPU time.	48
5-11	Convergence of the cost function as a function of design iterations and CPU time for the target lift coefficient case.	48
5-12	Convergence of the gradient of the cost function as a function of design iterations and CPU time for the target lift coefficient case.	49
5-13	The pressure distribution, normalized by the freestream pressure, over the initial and optimized airfoil for the $P = 4$ case with a mesh dimension of 12 x 12.	49

CHAPTER 1

Introduction

Computational Fluid Dynamics (CFD), a relatively new field, has gone through significant changes in the past decades fueled by the need for faster and more efficient algorithms to compute numerical solutions. As these algorithms have matured and gained accuracy, CFD has turned into a more prevalent analysis tool which has significantly decreased the time needed by engineers for validation. The additional coupling with optimization schemes has resulted in CFD truly revolutionizing the design process.

Traditional CFD programs rely on what are known as low-order methods. These are methods defined to have a spatial order of accuracy, the rate at which the error of the numerical solution decreases as the mesh is refined, of at most 2. This low order of accuracy in turn results in the need for very fine meshes for obtaining accurate numerical solutions with low errors. The drawback, however, is that solutions on these fine meshes are accompanied with very high computational costs.

This impediment caused by the order of accuracy of low-order methods has resulted into increased research into computational approaches known by, unsurprisingly, high-order methods. These methods allow for much higher spatial orders of accuracy, thus allowing the ability to obtain numerical solutions with low errors on coarser meshes. Several high-order methods have been the principal focus of research

in the past years including the Discontinuous Galerkin method, Spectral Difference method and Flux Reconstruction approach.

Design in aerospace settings had traditionally, before the advent of CFD, been an analytical process. Optimal solutions were arrived at by using mathematical solutions to simplified problems. The introduction of CFD, however, has resulted in the ability to solve complicated constrained optimization problems. These approaches have been developed extensively for use with gradient-based methods. In this approach to optimization, the gradient of a given cost function is required for the optimization, a property that, using traditional methods, is quite costly to compute. The adjoint approach, however, provides a remedy for this. Developed using control theory, the adjoint method allows for a very efficient calculation of the gradient by calculating it indirectly through solving what is known as the adjoint equation. Due to its efficient computation of gradients, the adjoint approach has not only successfully been used in aerospace applications such as in transonic design problems, it has also been developed extensively for commercial CFD programs.

1.1 Contribution

Although the adjoint approach for design has been studied in detail, these works have been concerned primarily with low-order methods. The investigation of this approach with the use of high-order methods however has not been explored. This, therefore, has been the principal focus of this work. The high-order method used in this study is the Discontinuous Galerkin method, and an adjoint solver has been paired with it. Simple two dimensional optimization cases have first been investigated to check the capabilities of high-order approaches when used in optimization

settings. The efficacy of high-order approaches is also investigated, by checking in detail whether the higher orders of accuracy provided by these approaches affords any advantage to optimization applications.

1.2 Thesis Overview

The following chapters are structured as follows. Chapter 2 introduces the Discontinuous Galerkin method. The equations and formulations of the method are first outlined, followed by a detailed description of how Bézier basis functions have been used in this work. Chapter 3 focuses on the optimization framework, where the adjoint approach is introduced. Chapter 4 presents the validation cases used in order to verify the correctness of the code and its implementation. Chapter 5 demonstrates results obtained from the high-order adjoint-based optimization process. Finally, Chapter 6 summarizes the key findings and outlines future work.

CHAPTER 2

The Governing Equations

In this chapter, the governing physical equations as well as their numerical discretization will be outlined. First, the two-dimensional Euler Equations will be presented, followed by a summary of the Discontinuous Galerkin (DG) method. In addition, special focus will be placed on describing the use of B-splines and the Bezier basis functions as a basis for analysis in Section 2.3.

2.1 2D Euler Equations

All cases studied in this work involve solutions to inviscid fluid flow problems in two dimensions. The mathematical equations governing such flow is known as the two-dimensional Euler equations and can be written as

$$\frac{\partial \vec{U}}{\partial t} + \frac{\partial \vec{F}}{\partial x} + \frac{\partial \vec{G}}{\partial y} = \mathbf{0}, \quad (2.1)$$

where the state $\vec{U}$ as well as the fluxes $\vec{F}$ and $\vec{G}$ are given by

$$\vec{U} = \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ \rho e \end{bmatrix}, \quad \vec{F} = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho ue + pu \end{bmatrix}, \quad \vec{G} = \begin{bmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ \rho ve + pv \end{bmatrix} \quad (2.2)$$

Equation (2.2) consists of five flow field variables where ρ corresponds to the density, u is the velocity in the x -direction, v is the velocity in the y -direction, e is the total energy per unit mass and p is the static pressure. An additional fifth equation is required and is taken to be the equation of state of a calorically perfect gas, given by

$$p = (\gamma - 1)\rho\left(e - \frac{u^2 + v^2}{2}\right), \quad (2.3)$$

where γ is the heat capacity ratio of the fluid.

2.2 The Discontinuous Galerkin Method

In this section, an overview will be given for how to apply the Discontinuous Galerkin method to solve problems on two dimensional domains. For a more detailed formulation of the method, the reader is referred to the textbook by Hesthaven and Warburton [1]. Consider solving the general 2D scalar conservation equation given by

$$\frac{\partial U}{\partial t} + \nabla \cdot \mathbf{F} = 0, \quad (2.4)$$

where $\mathbf{F} = (F, G)$, $F = F(U)$ and $G = G(U)$. Since each physical equation given in equation (2.1) and (2.2) has been written in its conservation form, the procedure outlined in what follows for solving equation (2.4) will be applicable to ultimately solving the Euler equations as well.

The numerical solution to equation (2.4) is sought over a computational domain Ω partitioned into N_e non-overlapping elements denoted by Ω^k , each with a surface denoted by Γ^k , such that

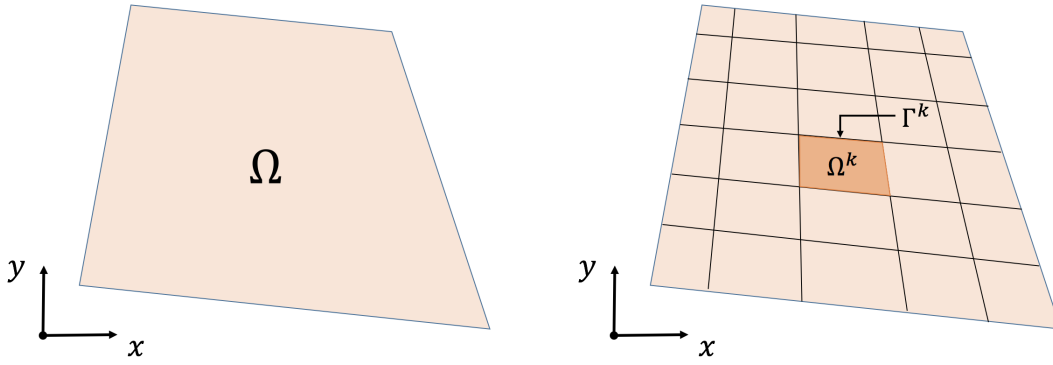


Figure 2–1: A schematic of the computational domain and the same domain when partitioned into elements.

$$\Omega = \bigcup_{n=1}^{N_e} \Omega^n. \quad (2.5)$$

Figure (2–1) shows an example of this initial and partitioned domain. With the domain separated, the solution to the partial differential equation (PDE) will be considered within each element. Let the exact solution to the PDE in equation (2.4) within the k^{th} element, Ω^k , be U_k^{exact} and the approximate numerical solution be U_k^h , where U_k^h is defined to be zero outside of element. Using the local solution from each element, the global approximate solution U^h to the PDE over the domain Ω can be written as

$$U^{exact} \approx U^h = \bigoplus_{n=1}^{N_e} U_n^h, \quad (2.6)$$

where U^{exact} is the total exact solution.

In the DG approach, rather than directly searching for the numerical solution over the whole domain, the solution on each individual element will be computed

separately. Furthermore, it is important to note that no restriction on continuity will be placed on the solutions between these elements and, as such, multiple solutions may be defined on element boundaries.

To begin, first form the residual given by

$$R^h(x, y, t) = \frac{\partial U^h}{\partial t} + \nabla \cdot \mathbf{F}(U^h) \approx 0. \quad (2.7)$$

The approximate solution U^h will not exactly solve equation (2.4) and as such the residual will be equal to a non zero value. The goal of the numerical approach will therefore be to search for the best approximate solution to the PDE that will, in turn, minimize the residual.

To do this, a finite set of basis functions, or test functions, will be used to represent the solution on each element. Consider the k^{th} element Ω^k . On this element, define N_p basis functions $w_k(x, y)$ which are identically zero outside of Ω^k . The numerical solution U_k^h on this element can then be represented as

$$U_k^h = \sum_{i=1}^{N_p} a_i^k(t) w_i(x, y), \quad (2.8)$$

where the unknowns to be solved now are the coefficients multiplying each basis function. To obtain the N_p independent equations needed to solve for the N_p unknown a_i values, a requirement is made that the residual must be orthogonal to all the basis functions. In mathematical form, this can be represented as,

$$\int_{\Omega^k} R^h(x, y, t) w_i(x, y) d\Omega = 0, \quad \forall i = 1, \dots, N_p \quad (2.9)$$

Substituting the expression for the residual from equation (2.7), one obtains

$$\int_{\Omega^k} \left(\frac{\partial U^h}{\partial t} + \nabla \cdot \mathbf{F}(U^h) \right) w_i(x, y) d\Omega = 0, \quad (2.10)$$

which can be expanded, integrated by parts, and simplified to yield

$$\int_{\Omega^k} w_i \frac{\partial U_k^h}{\partial t} d\Omega - \int_{\Omega^k} \nabla w_i \cdot \mathbf{F}(U_k^h) d\Omega + \int_{\Gamma^k} w_i (\mathbf{F}(U^h) \cdot \hat{n})^* d\Gamma = 0. \quad (2.11)$$

Equation (2.11) is referred to as the weak form of the DG scheme. It consists of two volume integral terms over the the element Ω^k and a surface integral term over the boundary Γ^k of the element. Furthermore, the surface integral term also consists of an expression $(\mathbf{F}(U^h) \cdot \hat{n})^*$ which is known as the numerical flux. This term arises due to the fact that the DG method allows for discontinuities in the solution between elements. Therefore, on a boundary shared by two elements, two possible values for the numerical solution U^h exist and, as such, a choice of one solution, or combination of these solutions, is needed to find the value in the integral. Equation (2.8) can next be substituted in place of U_k^h in equation (2.11) in the first integral and the summation term may be brought outside the integral to yield

$$\sum_{j=1}^{N_p} \frac{da_j^k}{dt} \int_{\Omega^k} w_i w_j d\Omega - \int_{\Omega^k} \nabla w_i \cdot \mathbf{F}(U_k^h) d\Omega + \int_{\Gamma^k} w_i (\mathbf{F}(U^h) \cdot \hat{n})^* d\Gamma = 0, \quad \forall i = 1, \dots, N_p \quad (2.12)$$

Equation (2.12) is a set of N_p linear equations that can be formed for each element. This linear system can be solved at each time step for the derivatives of the coefficients

a_j^k that form the approximation of the solution U_k^h within that element. The solution to the unsteady PDE in equation (2.4) can now be computed at each discrete time step. This is done by first using a given initial condition to the PDE to obtain the initial values for a_i^k in equation (2.8) for the approximate solution U_k^h in each element. Then, the system of equations in (2.12) is assembled at the given time step for each element and it is solved to obtain the derivatives $\frac{da_i^k}{dt}$ of each coefficient. With the values of the derivatives, a time stepping scheme (such as a Runge Kutta or Forward Euler scheme) may be used to obtain the values of the coefficient for the next time step and the process is repeated.

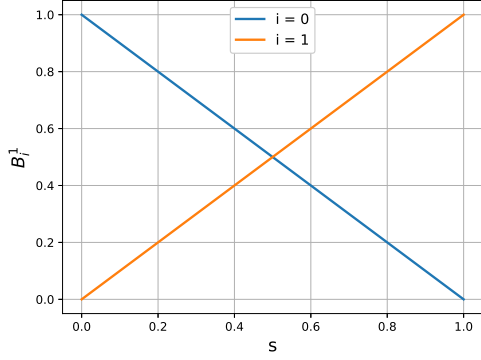
To solve the Euler equations listed in Section 2.1, an analogous approach is used. This can be done by representing the approximate solution in the element Ω^k on the computational domain as

$$\vec{U}_k^h = \sum_{i=1}^{N_p} \vec{a}_i^k(t) w_i(x, y), \quad (2.13)$$

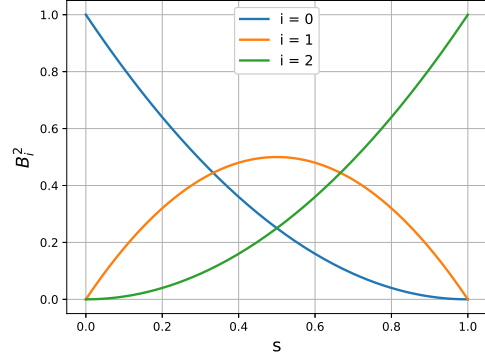
where the vector $\vec{a}_i^k(t)$ has the same number of components as physical equations. Moreover, the choice of which numerical flux to use depends primarily on the characteristics of the equations being solved. For solving the Euler equations in this study, the Roe flux has been employed for all cases.

2.3 Bézier Functions as a Basis for Analysis

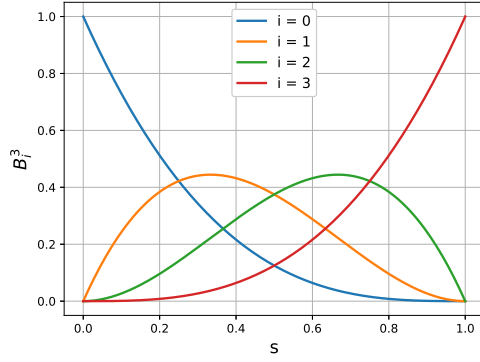
In this work, the Bézier basis functions have been used to represent the numerical solution in all cases. In one dimension, for a given order P , there are $P + 1$ basis functions of the form



(a) $P = 1$



(b) $P = 2$



(c) $P = 3$

Figure 2–2: Plots of the 1D Bézier basis functions for $P = 1$ (a), $P = 2$ (b) and $P = 3$ (c).

$$B_i^P(s) = \binom{P}{i} s^i (1-s)^{P-i}, \quad s \in [0, 1], \quad i = 0, \dots, P. \quad (2.14)$$

Each basis is therefore defined on the domain $[0, 1]$. Moreover, in two dimensions, for a given order P , there are $(P + 1)^2$ basis functions each defined as

$$B_{i,j}^P(s, t) = B_i^P(s) B_j^P(t), \quad s, t \in [0, 1]. \quad (2.15)$$

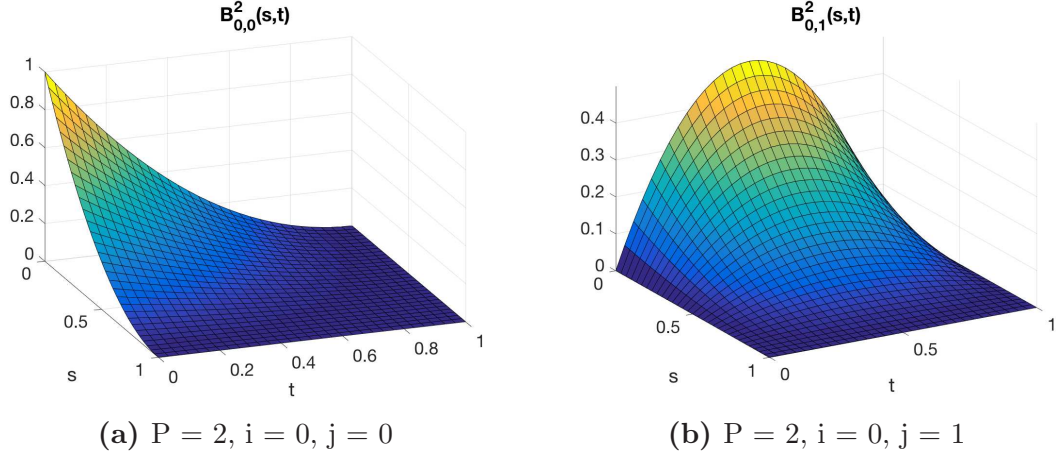


Figure 2–3: Plots of the 2D Bézier basis functions for $P = 2$. 2 of the total 9 basis functions are shown. The function for $i = 0, j = 0$ is shown in (a) and that for $i = 0, j = 1$ is shown in (b).

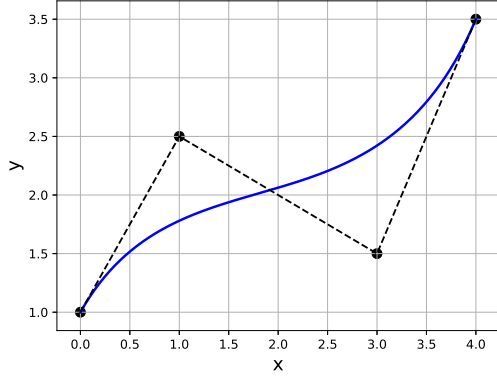
Each function is now a surface defined on the s, t domain. The two dimensional formulation of the basis functions as can be seen is simply defined as the product of two one dimensional basis functions but using two distinct parameters. Plots of 1D and 2D Bézier basis functions are shown in Fig. (2–2) and Fig. (2–3).

With the basis functions defined, it is now possible to introduce the Bézier curve. A Bézier curve is a parametric curve that requires, given an order P , $P + 1$ basis functions and $P + 1$ control points $\vec{x}_i$, where $\vec{x}_i = (x_i, y_i)$ in the 2D case. The curve is then defined as

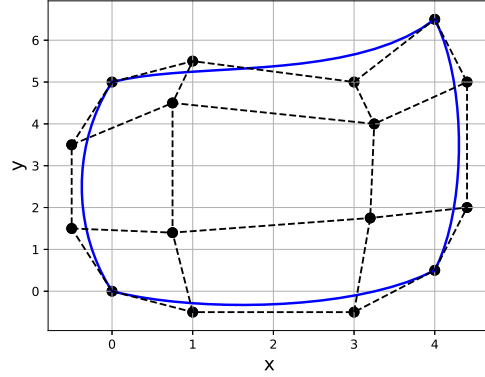
$$\vec{Q}(s) = \sum_{i=0}^P B_i^P(s) \vec{x}_i. \quad (2.16)$$

The free parameter s for any Bézier curve is, just like in the case of the basis functions, defined on the domain $[0, 1]$. The Bézier curve has several key properties, including

- The curve always interpolates the first and last control points ($\vec{x}_0$ and $\vec{x}_P$).



(a) A Bézier Curve, $P = 3$



(b) A Bézier Patch, $P = 3$

Figure 2–4: Plots of a sample Bézier Curve (a) and Bézier Patch (b) for $P = 3$. The control points for both are shown as the black points. Note that $P + 1 = 4$ control points are present for the case of the curve, and there is a lattice of $(P + 1)^2 = 16$ control points for the case of the patch. In (b), the outer boundary of the patch has been shown in blue.

- All Bezier curves are enclosed within the convex hull of its control polygon.
- The displacement of any control point that defines the curve will result in a global deformation. That is, the effect of the control points are not localized.

Similarly, in two dimensions, a lattice of structured control points are needed, each with a given i and j index. This lattice of control points, along with the two dimensional basis functions, result in a parametric surface known as a Bézier patch and given by

$$\vec{Q}(s, t) = \sum_{i=0}^P \sum_{j=0}^P B_i^P(s) B_j^P(t) \vec{x}_{ij}. \quad (2.17)$$

Shown in Fig. (2–4) are examples of a 1D Bézier along and a 2D Bézier patch.

2.3.0.1 The Reference Element

To efficiently obtain the numerical solution to the PDE on the physical domain using the DG method, a transformation is commonly used. This transformation, unique for each element, maps each given element on the physical domain onto a common reference element on a domain denoted using the coordinates ξ and η . Moreover, this reference element is defined on the domain $\{(\xi, \eta) \mid -1 \leq \xi, \eta \leq 1\}$. Shown in Fig. (2–5) is a schematic diagram showing this mapping of an arbitrary physical element onto the reference element. The basis functions and PDE are then mapped onto this reference element as well, and the numerical solution is computed there.

The transformation function used for the mapping for two dimensional cases is given as

$$\vec{x}(\xi, \eta) = \mathbf{T}(\xi, \eta) = \sum_{i=1}^{(P+1)^2} w_i(\xi, \eta) \vec{x}_i, \quad (2.18)$$

where $w_i(\xi, \eta)$ is a collection of basis functions (in this work the Bézier basis functions) mapped onto the reference element. In addition, along with the basis functions, a set of $(P+1)^2$ control points $\vec{x}_i$ are used to define the geometry for each element. Thus, a distinct mapping function is used for each element to transform onto the same reference element, since the set of control points $\vec{x}_i$ are in general distinct for each element. For the case of the Bézier basis, each element is treated as a Bézier patch, and the control points $\vec{x}_i$ are the lattice of control points needed to define the patch. With the mapping in place, the PDE is transformed as well, and this transformed PDE is what is solved to obtain the local solutions U_k^h on the elements.

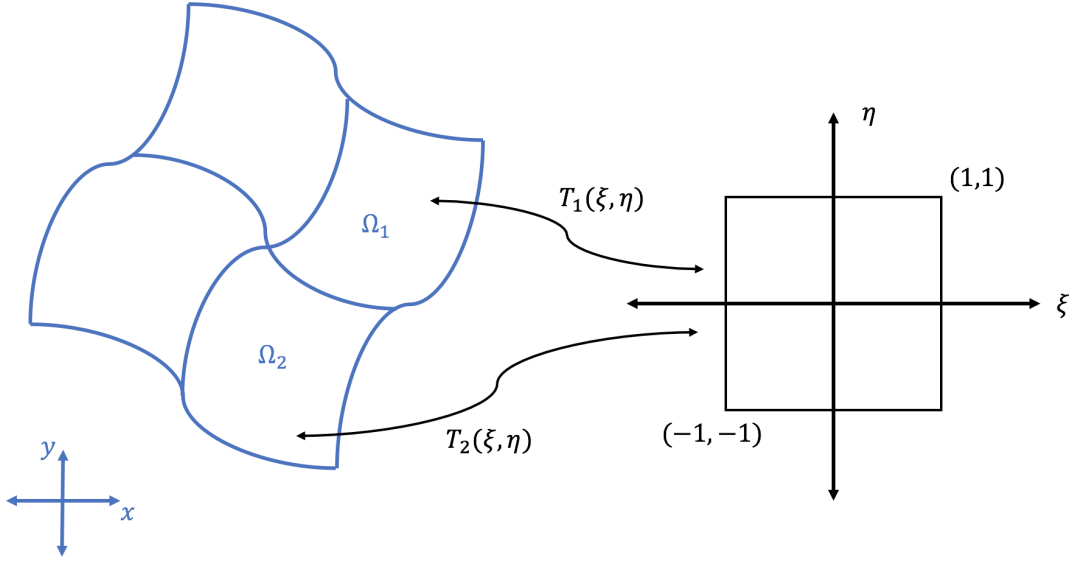


Figure 2–5: A schematic showing the mapping from the physical space to the common reference element. Note, each element on the computational domain, Ω_i , has a unique mapping denoted by $T_i(\xi, \eta)$

When the PDE is numerically solved, all information about the geometry is ultimately included within these mapping functions. Note, however, that in equation (2.18) the choice of the basis functions $w_i(\xi, \eta)$ used in the mapping are the Bézier basis functions. For the actual geometry (for instance the geometrical CAD model); different families of basis functions can be used. When this is the case, an approximation for the physical boundary must be made using the functions $w_i(\xi, \eta)$ and the PDE is then solved on this approximate domain.

2.3.0.2 B-Splines

All geometries used in this work are represented using B-spline curves (also referred to simply as B-splines). The B-spline basis function is a generalization of

the Bézier basis functions presented in Section 2.3. Consider the case where n B-spline basis functions are desired, each having an order P . To define each basis function, first a knot vector is specified of the form

$$\mathbf{t} = [t_0, t_1, \dots, t_{n+P+1}], \quad (2.19)$$

where the scalars t_i in the vector are placed in ascending order. With the knot vector defined, the i_{th} B-spline basis function can then be defined recursively as

$$N_{i,P}(t) = \frac{t - t_i}{t_{i+P} - t_i} N_{i,P-1}(t) + \frac{t_{i+P+1} - t}{t_{i+P+1} - t_{i+1}} N_{i+1,P-1}(t), \quad (2.20)$$

where

$$N_{i,0}(t) = \begin{cases} 1 & t_i \leq t < t_{i+1} \\ 0 & \text{otherwise} \end{cases} \quad (2.21)$$

and any $\frac{0}{0}$ term is defined to be zero. With the definition of the 1D basis, the extension to obtain the 2D basis functions is straightforward and follows the same procedure as the Bézier basis. Moreover, using a set of n control points with the n basis functions, it is possible to define B-splines identical to how the Bézier curve in equation (2.16) is defined.

Due to the close relationship between the B-spline basis and Bézier basis, it is possible to convert geometries represented using B-splines to ones using Bézier curves using a process known as Bézier extraction [3]. Using this operator, the following is

the procedure that has been used to generate the computational mesh around the 2D geometries defined by B-splines in this work:

1. The B-spline geometry is first processed by applying the Bézier extraction operator. This results in a sequence of Bézier curves defining the geometrical model.
2. Each Bézier curve defining the geometry is then extended into a Bézier patch. In addition to this set of patches, many more patches are added to the domain as well.
3. The collection of Bézier patches is treated as the mesh for computing the numerical solution. Furthermore, each patch is considered as a distinct element in the mesh.

Once all patches have been created, the DG method is then used to compute the numerical solution on the given computational domain.

CHAPTER 3

Optimization Framework

Single-objective aerodynamic shape optimization was the focus of this work and this chapter formulates the approach employed to perform the optimization. The adjoint approach to optimization is first introduced, followed by an explanation of the mesh deformation algorithm. Finally, the implementation of the quasi-Newton method as a means to find the local minimum of the given objective function is discussed.

3.1 The Adjoint Approach

Aerodynamic shape optimization requires two principal items: a cost function and a set of design variables. The cost function, also referred to as the objective function, is a function that is required to be minimized. For the purposes of aerodynamic shape optimization, this cost function for instance could be the drag coefficient. The design variables are utilized to adjust the geometry of the body. The goal of the optimization then becomes a simple one: to find the location of the design variables such that the cost function attains a minimum value.

Consider the case now where a cost function I is to be minimized using a set of N_d design variables $\boldsymbol{\alpha}$ where $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \dots, \alpha_{N_d}]$. In order to minimize the cost function, its gradient with respect to these N_d design variables is needed:

$$\nabla_{\alpha} I = \left[\frac{\partial I}{\partial \alpha_1}, \dots, \frac{\partial I}{\partial \alpha_{N_d}} \right]. \quad (3.1)$$

With the value of the gradient, the design variables can then be moved in the negative gradient direction to find an updated location such that I decreases as follows

$$\boldsymbol{\alpha}^{n+1} = \boldsymbol{\alpha}^n - \lambda \nabla_{\alpha} I, \quad (3.2)$$

where λ is a scalar value for the step length and the values α^n and α^{n+1} represent the locations of the design variables at the n and $n + 1$ design iterations. If this process is repeated for each iteration, the optimal location of the design variables will eventually be attained.

Central to the optimization process is the ability to accurately and efficiently compute the gradient of the objective function. One approach to compute each component of the gradient could be using finite differences:

$$\frac{\partial I}{\partial \alpha_i} = \frac{I(\alpha_i + \Delta \alpha_i) - I(\alpha_i)}{\Delta \alpha_i}. \quad (3.3)$$

Although this approach may be looked upon favorably due to its simplicity, it however becomes an unfeasible option as the number of design variables grow. This is because, for each finite difference computation, a new flow solve must be computed followed by an evaluation of the objective function. As each flow solve can be very expensive, the computation of the gradient using this approach would not be practical. An alternate methodology to compute the gradients is needed, and the adjoint approach is one such method.

The adjoint approach allows for the ability to compute the gradients of the objective function with a computational cost as expensive as one flow solve. Consider the objective function $I(\mathbf{w}, \boldsymbol{\alpha})$, where I is a function of the N_f flow variables $\mathbf{w}$ and the N_d design variables $\boldsymbol{\alpha}$. These flow variables must additionally satisfy the flow constraint equations

$$\mathbf{R} = \mathbf{R}(\mathbf{w}, \boldsymbol{\alpha}) = 0. \quad (3.4)$$

The flow constraint equations, $\mathbf{R}$, is a vector-valued function with N_f components that consists of the physical equations, in this case the Euler equations, in their discretized form.

Consider now a variation in the shape $\delta\boldsymbol{\alpha}$. This, in turn, induces a variation in the flow, represented as $\delta\mathbf{w}$. The total variation in the cost function due to perturbation in the geometry can then be given by

$$\delta I = \left[\frac{\partial I}{\partial \mathbf{w}} \right]^T \delta \mathbf{w} + \left[\frac{\partial I}{\partial \boldsymbol{\alpha}} \right]^T \delta \boldsymbol{\alpha}. \quad (3.5)$$

The total variation of the flow constraint equations can also be computed as

$$\delta \mathbf{R} = \left[\frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right]^T \delta \mathbf{w} + \left[\frac{\partial \mathbf{R}}{\partial \boldsymbol{\alpha}} \right]^T \delta \boldsymbol{\alpha} = 0. \quad (3.6)$$

Since the variation in the governing equations in equation (3.6) is 0, it may be multiplied by a Lagrange multiplier $\boldsymbol{\psi}^T$ and subtracted from equation (3.5). Subtracting this term, and then rearranging terms, yields

$$\delta I = \left[\left[\frac{\partial I}{\partial \mathbf{w}} \right]^T - \boldsymbol{\psi}^T \left[\frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right] \right] \delta \mathbf{w} + \left[\left[\frac{\partial I}{\partial \boldsymbol{\alpha}} \right]^T - \boldsymbol{\psi}^T \left[\frac{\partial \mathbf{R}}{\partial \boldsymbol{\alpha}} \right] \right] \delta \boldsymbol{\alpha}. \quad (3.7)$$

The dependence of δI on $\delta \mathbf{w}$ can then be eliminated by computing the multiplier $\boldsymbol{\psi}$ that satisfies

$$\left[\frac{\partial I}{\partial \mathbf{w}} \right]^T = \boldsymbol{\psi}^T \left[\frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right]^T. \quad (3.8)$$

Using this value for $\boldsymbol{\psi}$, the variation in the objective can be computed using

$$\delta I = \mathbf{G} \delta \boldsymbol{\alpha}, \quad (3.9)$$

where $\mathbf{G}$ is equivalent to the gradient of the cost function $\nabla_{\boldsymbol{\alpha}} I$ and is given by

$$\mathbf{G} = \left[\frac{\partial I}{\partial \boldsymbol{\alpha}} \right]^T - \boldsymbol{\psi}^T \left[\frac{\partial \mathbf{R}}{\partial \boldsymbol{\alpha}} \right]. \quad (3.10)$$

Equation (3.8) is known as the adjoint equation while equation (3.10) is known as the sensitivity equation. The solution of equation (3.8), the most expensive step in this procedure, requires as much time as only one flow solve.

3.2 Radial Basis Function Mesh Deformation

Upon the deformation of the geometry after a given design cycle, an efficient mesh deformation algorithm is needed that does not significantly deteriorate the mesh quality during the optimization. For all cases in this study, a mesh deformation scheme using Radial Basis Functions (RBFs) has been used.

RBF first consists of selecting a set of N_{rp} mesh points, referred to as RBF points. The scheme can then interpolate the displacement of a volume mesh point located at $\mathbf{x}$ using

$$s(\mathbf{x}) = \sum_{i=1}^{N_{rp}} \alpha_i \phi(\|\mathbf{x} - \mathbf{x}_{ri}\|), \quad (3.11)$$

where $\mathbf{x}_{ri}$ are the positions of the RBF mesh points, α_i are a set of coefficients and ϕ is the basis function used in the interpolation. The value of the coefficients may be determined by imposing the condition that at the RBF points, their known displacements should be recovered.

The use of matrices simplifies the implementation of the RBF scheme when the entire set of mesh points are considered together. First, the coefficients α_i must be computed by using the known displacements of the RBF points $\Delta\mathbf{x}_r$, where $\Delta\mathbf{x}_r$ is a matrix consisting of N_{rp} rows and, in the 2D case, 2 columns for the displacements of the x and y position of each point. Using the notation $\phi_{r_i r_j} = \phi(\|\mathbf{x}_{ri} - \mathbf{x}_{rj}\|)$ for the i^{th} and j^{th} RBF points, a system of equations can be obtained of the form

$$\Delta\mathbf{x}_r = \mathbf{M} \boldsymbol{\alpha}, \quad (3.12)$$

where the matrix $\mathbf{M}$, of dimension $N_{rp} \times N_{rp}$, is of the form

$$\mathbf{M} = \begin{bmatrix} \phi_{r_1 r_1} & \phi_{r_1 r_2} & \cdots \\ \phi_{r_2 r_1} & \phi_{r_2 r_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}. \quad (3.13)$$

Once the values of the coefficients $\boldsymbol{\alpha}$ are evaluated, the displacement of all volume mesh points can then be computed as

$$\Delta \mathbf{x}_v = \mathbf{A} \boldsymbol{\alpha} = \mathbf{A} \mathbf{M}^{-1} \Delta \mathbf{x}_r \quad (3.14)$$

where

$$\mathbf{A} = \begin{bmatrix} \phi_{v_1 r_1} & \phi_{v_1 r_2} & \cdots \\ \phi_{v_2 r_1} & \phi_{v_2 r_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}. \quad (3.15)$$

Analogous to the case of the $\mathbf{M}$ matrix, the notation $\phi_{v_i r_j} = \phi(\|\mathbf{x}_{v_i} - \mathbf{x}_{r_j}\|)$ has been used here, where v_i refers to the i^{th} volume point and r_j corresponds to the j^{th} RBF point. If there are N_{vp} volume points, then matrix $\mathbf{A}$ is of dimension $N_{vp} \times N_{rp}$. Moreover, $\Delta \mathbf{x}_v$, of dimension $N_{vp} \times 2$ in the 2D case, is the displacement of the coordinates of each volume point.

Finally, the choice of the RBF basis function $\phi(\xi)$ is important as it will dictate the mesh quality and matrix conditioning. In all cases presented in this study, Wendland's C^0 basis function, given by

$$\phi(\xi) = (1 - \xi)^2, \quad \phi(\xi) = 0, \quad \forall \quad \xi > 1, \quad (3.16)$$

has been the function that has been used.

Fig. (3-1) shows an example of an RBF deformation on an initially rectangular mesh.

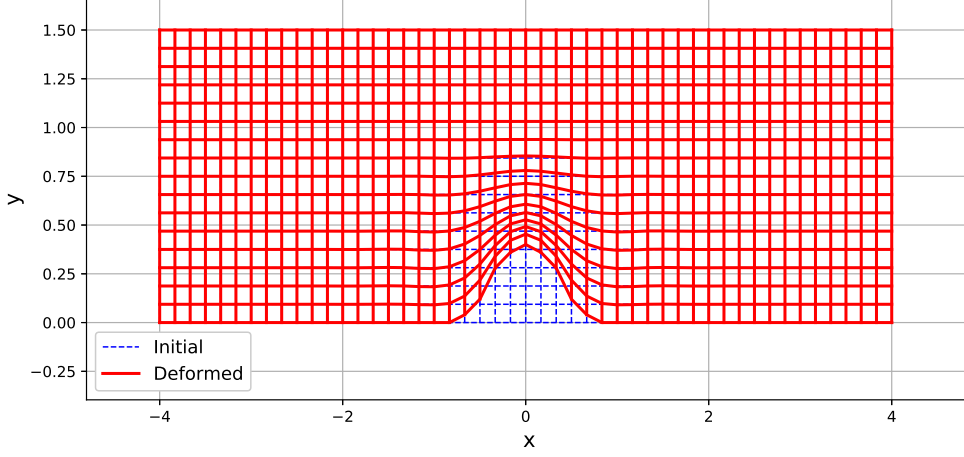


Figure 3–1: An example of an RBF mesh deformation where the Wendland C^0 basis function has been used. All points around the mesh are considered as RBF points, and all interior points are the volume mesh points.

3.3 Optimization Strategy

All optimization cases presented in this study are solutions to unconstrained problems of the form

$$\min_{\mathbf{x}_\alpha} I(\mathbf{x}_\alpha), \quad \mathbf{x}_\alpha \in \mathbb{R}^n, \quad (3.17)$$

where I is the cost function, $\mathbf{x}_\alpha$ are the design variables and n is equivalent to the number of design variables multiplied by the number of degrees of freedom each one has. These design variables are the control points of the B-spline curve that formed the surface of the geometry to be optimized. Moreover, at each design iteration, the Qausi-Newton method can be used to obtain the search direction given by

$$\mathbf{p}_k = -\mathbf{B}_k^{-1} \nabla I, \quad (3.18)$$

where $\mathbf{p}_k$ is the search vector and ∇I is the gradient of the cost function with respect to the design variables. The matrix $\mathbf{B}_k$ in equation (3.18) is an approximation for the Hessian of the cost function, which in this study is computed using the Broyden-Fletcher-Goldfarb-Shanno (BFGS) approach. Finally, the step length λ_k can be computed using a backtracking line search approach. With the search direction and step length determined, the updated positions of the design variables for each design iteration is then determined as

$$\mathbf{x}_\alpha^{n+1} = \mathbf{x}_\alpha^n + \lambda_k \mathbf{p}_k. \quad (3.19)$$

The procedure is then repeated until the optimal solution is obtained.

In the cases studied, instabilities quickly arose when the number of design variables increased. Large numbers of design variables resulted in search vectors $\mathbf{p}_k$ that created kinks in the geometry and, as the shape lost its smoothness, these kinks would grow to yield unstable solutions. To ensure the smoothness of the geometry, an explicit smoothing operator was applied on the search direction. Laplacian smoothing, a mesh smoothing algorithm, is a process used commonly to remove noise from a mesh that forms a given object while retaining its basic overall shape. This is the approach that has been used in this work to smooth the search directions. Using a smoothing matrix given by

$$\mathbf{L} = \begin{bmatrix} -1 & \frac{1}{2} & 0 & 0 & \dots & 0 \\ \frac{1}{2} & -1 & \frac{1}{2} & 0 & \dots & 0 \\ 0 & \frac{1}{2} & -1 & \frac{1}{2} & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \end{bmatrix}, \quad (3.20)$$

the updated search direction can be obtained using

$$\mathbf{p}_k^{n+1} = \mathbf{p}_k^n + \lambda \mathbf{L} \mathbf{p}_k^n, \quad (3.21)$$

where the λ is a scalar satisfying $0 < \lambda < 1$. Multiple iterations of equation (3.21) may be applied, where each iteration results in more and more noise being removed from the search direction. Use of this smoothing operator ultimately allows for the body being optimized to retain its smoothness between design iterations.

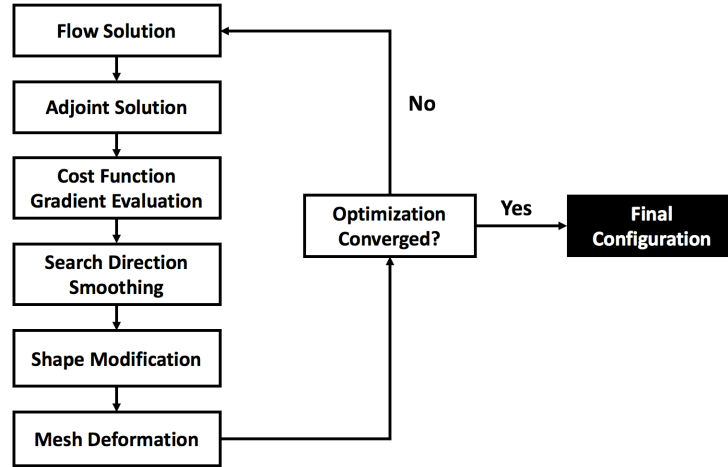


Figure 3–2: A summary of the optimization design cycle.

Figure (3–2) shows a summary of the overall optimization design cycle. Each cycle consists first of a flow solve, which in this work is completed using the high-order Discontinuous Galerkin method. Next, the adjoint equation is solved and the solution from that equation is substituted into the sensitivity equation to obtain the gradient of the cost function with respect to the design variables. Once the gradient has been computed, the search direction as well as the step length are obtained using the quasi-Newton method and a backtracking algorithm. The search vector $\mathbf{p}_k$ is then modified using an explicit smoother in order to retain the smoothness of the geometry. Using the smoothed direction, the new positions of the design variables are computed, and this, in turn, modifies the shape of the object itself. The RBF mesh mover is then used to perturb the mesh around the object. If convergence to the optimum shape has then been obtained, the algorithm exits the design loop. If not, the process is repeated.

CHAPTER 4

Validation Cases

This chapter includes the cases that were used to validate the flow solver as well as optimizer. A mesh refinement study is first introduced for a canonical case of flow through a channel with a bump, and the orders of accuracy of the scheme are verified. Moreover, the implementation of the objective function for the inverse design optimization cases that are presented in Chapter 5 is also validated using a refinement study.

4.1 Orders of Accuracy

4.1.1 The Gaussian Bump Case

The case that is considered to validate the orders of accuracy is for the flow through an inviscid 2D channel. This channel consists of a straight upper face, while on the lower face a bump is introduced. The bump profile that has been used for the validation cases is given by a Gaussian function which is of the form

$$f(x) = ae^{-\frac{(x-b)^2}{2c^2}}. \quad (4.1)$$

In addition, for the bump in the channel presented in this section, the values for a , b and c are given by $a = 0.30$, $b = 0.0$ and $c = 0.6$.

The geometry for this case needs to further be expressed using B-splines. The upper face provides no issues for this, as it is simply defined by a horizontal line and

can thus be represented easily with a spline. The lower surface, on which the bump resides, requires more steps in the generation of the B-spline. To create this spline, a continuous least squares approximation is used to locate the positions to place the control points. This step is crucial to attain the orders of accuracy as it ensures that the mesh itself in the refinement study converges to the same given bump profile.

Once the B-spline boundary is defined, it is possible to generate the mesh by utilizing the Bézier extraction operator first followed by a placement of the Bézier patches to form the elements of the mesh. All internal elements, that is, those not on a wall boundary, have straight faces on all sides. Furthermore, these elements have their vertices perturbed using a Gaussian function as well, where the value of a in equation (4.1) has been decreased linearly to 0 as the vertices move closer to the upper surface. This results ultimately in a smoother distribution of elements within the channel. In addition, elements along the wall have faces that are curved in order to enable a higher order description of the geometry. The internal geometry nodes for these elements, those not on the element boundary, are positioned using a blending function algorithm proposed by Gordon and Hall [4]. This function places the internal nodes for these elements using the curvature data from the boundary faces.

A point of note should also be made for the fact that the mesh used for the inviscid channel convergence test is superparametric. This is related primarily to the transformation function described in Section 2.3 and represented in equation (2.18), which is restated here for convenience.

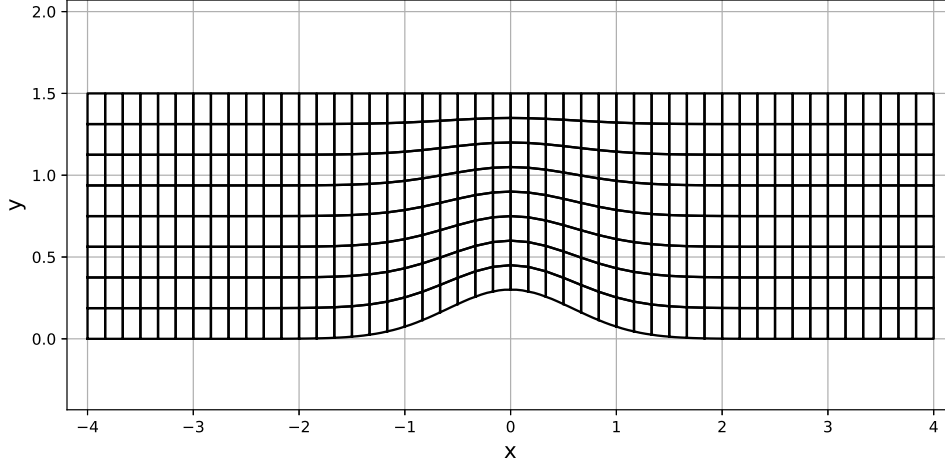


Figure 4–1: A plot of the $P_G = 3$, 48×8 (48 elements along the x direction and 8 along the y direction) mesh for the Gaussian function bump orders of accuracy tests.

$$\vec{x}(\xi, \eta) = \mathbf{T}(\xi, \eta) = \sum_{i=1}^{(P+1)^2} w_i(\xi, \eta) \vec{x}_i.$$

The basis functions $w_i(\xi, \eta)$ used to represent the mapping in equation (2.18) need not be of the same order as those used to represent the solutions. In the case where the order of the basis function for the mapping is equivalent to those used for the numerical solution, the elements are said to be isoparametric, whereas if the order is greater then they are referred to as being superparametric. In the cases presented in this section, superparametric elements have been used, where the order of the basis functions used to parametrize the geometry, denoted henceforth by P_G , is one greater than that those used to represent the solution. This, in turn, allows for a more accurate parametrization of the channel.

Fig. (4-1) shows a sample mesh with 48 elements along the x direction and 8 elements along the y direction that is used for the order of convergence study. Moreover, the mesh shown is made of elements with $P_G = 3$.

4.1.2 Boundary Conditions

The cases shown in this section considers subsonic flow through an inviscid channel. A slip wall boundary condition is placed first on the faces of the channel. On the sample mesh in Fig. (4-1), this corresponds to the upper and lower faces of the mesh. A total temperature, T_T , and total pressure, p_T , are further specified along on the leftmost boundary of the mesh. Moreover, a fixed back pressure is specified on the right boundary of the mesh with a value of $p_{back} = 0.99p_T$, resulting in fluid flowing from the left to right.

4.1.3 Results

For the high-order cases considered, the error should converge on the order of $O(h^{P+1})$, where h is a measure of the mesh size. This order of convergence has been verified by measuring the normalized entropy error, given as

$$\|s\|_2 = \left[\frac{1}{|\Omega|} \int_{\Omega} \left(\frac{\frac{p}{\rho^\gamma} - \frac{p_T}{\rho_T^\gamma}}{\frac{p_T}{\rho_T^\gamma}} \right) d\Omega \right]^{\frac{1}{2}}, \quad (4.2)$$

where Ω is the total computational domain, $|\Omega|$ is the volume of this domain and p_T and ρ_T are, respectively, the total pressure and density. For the cases shown, a 6 x 1 mesh is used initially and this mesh is then refined by doubling the number elements along the x and y direction. The metric used for the measure of the mesh size h is given as

Table 4–1: Normalized entropy error convergence for $P = 1, 2, 3$ using superparametric elements with $P_G = P + 1$.

Polynomial	Mesh Dimension	L_2 Error Entropy	Conv. Order
$P = 1$	6 x 1	3.2789e-04	-
	12 x 2	1.5507e-04	1.0803
	24 x 4	6.5573e-05	1.2417
	48 x 8	2.0659e-05	1.6663
	96 x 16	4.9798e-06	2.0526
	192 x 32	1.0190e-06	2.2889
$P = 2$	6 x 1	1.2318e-04	-
	12 x 2	3.0624e-05	2.0080
	24 x 4	7.1652e-06	2.0956
	48 x 8	1.3036e-06	2.4585
	96 x 16	1.8145e-07	2.8449
	192 x 32	2.1070e-08	3.1063
$P = 3$	6 x 1	3.3788e-05	-
	12 x 2	3.2789e-06	3.3652
	24 x 4	5.0553e-07	2.6973
	48 x 8	5.0253e-08	3.3305
	96 x 16	3.8174e-09	3.7185
	192 x 32	2.5608e-10	3.8979

$$h = \left(\frac{1}{n_{dof}} \right)^{\frac{1}{2}}, \quad (4.3)$$

where n_{dof} is equal to the total number of degrees of freedom and is given by, using the total number of elements N_e , $n_{dof} = N_e (P + 1)^2$. Finally, the solver used in the cases shown utilizes an implicit scheme using the analytical linearization of the discretized Euler equations. This solver iteratively computed the numerical solution until the residual converged below $1e - 14$.

Table (4-1) provides a summary of the error convergence results. It can be seen that the optimal convergence order of $P + 1$ is obtained for the values of P that were tested. A plot of the pressure contour for one of these cases is also shown in Fig. (4-2).

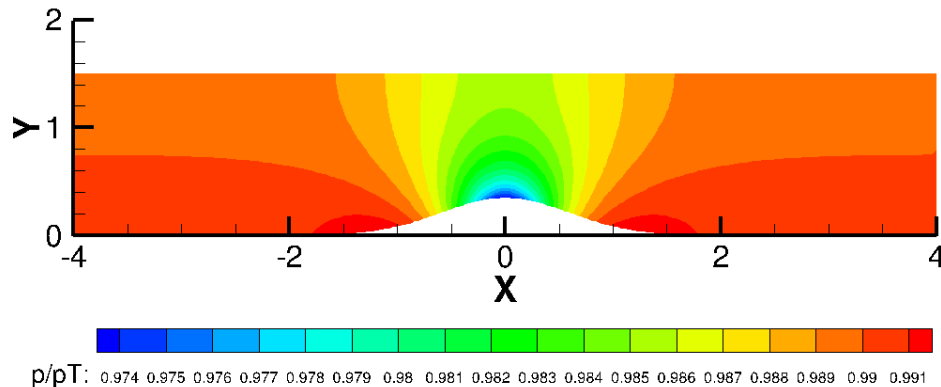


Figure 4-2: A plot of the pressure distribution normalized by the total pressure for the $P = 2$, 96×16 case

4.2 Inverse Design Cost Function Convergence

In addition to verifying the orders using the entropy error, an additional validation case is also presented here using the convergence of the inverse design cost function (also referred to as an objective function). This cost function is used in the inverse design optimization cases, where a desired target pressure distribution $p_{target}(s)$ is specified and the surface is optimized to meet this target. The cost function for this optimization case is given as

$$I = \frac{1}{2} \int_S (p(s) - p_{target}(s))^2 ds,$$

Table 4–2: Inverse design cost function convergence for $P = 1, 2, 3$ using superparametric elements with $P_G = P + 1$. The target mesh with a dimension of 96×16 was fixed for each P value.

Polynomial	Mesh Dimension	Cost Function	Conv. Order
P = 1	6 x 1	2.2911e-05	-
	12 x 2	1.1765e-05	0.9616
	24 x 4	2.4719e-06	2.2507
	48 x 8	2.7437e-07	3.1715
P = 2	6 x 1	1.5085e-05	-
	12 x 2	1.6390e-06	3.2023
	24 x 4	4.9776e-08	5.0412
	48 x 8	1.2361e-09	5.3316
P = 3	6 x 1	3.2587e-06	-
	12 x 2	1.9441e-08	7.3891
	24 x 4	6.5994e-10	4.8806
	48 x 8	7.8219e-12	6.3987

and is covered in more detail in Chapter 5. The given cost function should converge at $O(h^{2P+1})$, and this rate of convergence is measured to test the validity of not only the flow solver, but also the implementation of the function itself.

The target pressure distribution used for this convergence study corresponds to the pressure over a bump in an inviscid channel where $a = 0.35$ in equation (4.1). The design mesh, moreover, considers a slightly smaller bump with a value of $a = 0.25$. All other parameters, b and c , are kept fixed. The target mesh dimension is kept fixed as well at 96×16 for all cases.

Table (4–2) presents the results for the convergence rate of the objective function. It can be seen that the rate of convergence is indeed on the order of $O(h^{2P+1})$ for the cases for $P = 1$ and 2 . The convergence rate is approximately $2P + 1$ for the

$P = 3$ case, however the slight deviation will be investigated in more detail in future work.

CHAPTER 5

Optimization Results

This chapter presents three optimization cases that have been implemented using the high-order solver coupled with the adjoint approach. Two inverse design cases are first introduced followed by a target lift coefficient optimization.

5.1 Inviscid Channel Inverse Design

The inviscid channel with a bump described by a Gaussian function, presented in Chapter 4, is used as the first inverse design optimization case. This design problem consists of having a given geometry deform from its initial shape to a desired target shape. This is done by specifying a constant target pressure distribution, $p_{target}(s)$, to be achieved, which is given by the pressure over the target shape. The optimization problem can then be formulated using the cost function

$$I = \frac{1}{2} \int_S (p(s) - p_{target}(s))^2 ds, \quad (5.1)$$

where $p(s)$ is the pressure distribution over the surface that is undergoing the optimization. The successful minimization of the function in equation (5.1) will result in the optimized geometry attaining the shape of the target profile.

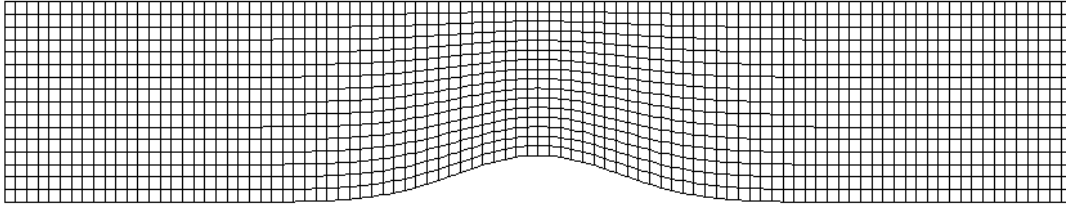
For the inviscid channel case, the target profile used was for a channel with a bump on its lower surface represented by the Gaussian function in equation (4.1) with a value of $a = 0.3$, $b = 0.0$ and $c = 0.6$. The target mesh also consisted of

96 elements along the x direction and 16 along the y direction. The flow is initially solved over this mesh, and then the pressure distribution obtained is treated as the value for $p_{target}(s)$ in the cost function for the optimization.

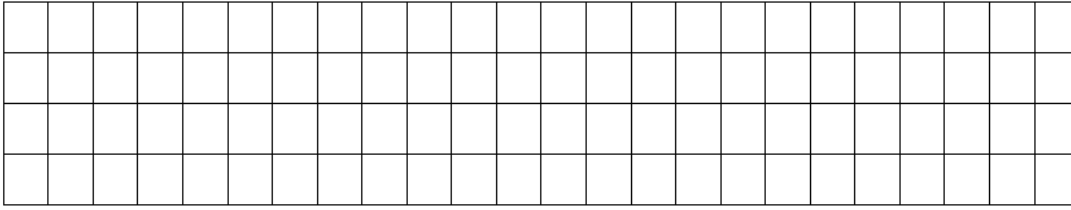
For the optimization, an initial channel profile is needed. In this case, the channel is initially completely flat, that is, no bump is present. By then specifying S in equation (5.1) to be the lower surface of this channel, minimization of the cost function allows for the bump to be generated. A coarser mesh is used for the design mesh (the mesh that is optimized) due to the high computational cost required for the optimization. As such, this mesh is of dimension 24×4 . Moreover, both the target and initial mesh are constructed using superparameteric meshes using basis functions of order $P_G = 3$ to specify the geometry. The mesh for the initial and target profiles are shown in Fig. (5-1).

This case has been run using the identical boundary conditions listed in Section 4.1.2. In addition, for the optimization loop, the design iterations stop when the gradient of the objective function is below $1e - 9$. Moreover, the design points used are the B-spline control points that describe the surface of the channel. For the optimization, these points are given one degree of freedom, as they are allowed to move in the y -direction. In total, 35 B-spline control points have been used for the inviscid channel case presented. Finally, for the high-order case considered, the order of the basis functions used is $P = 2$.

Fig. (5-2) shows the initial, optimized and target profile for the optimization case. The target bump profile, as can be seen, is quite well approximated. Near the peak of the bump a small discrepancy is observed, which is primarily a result of



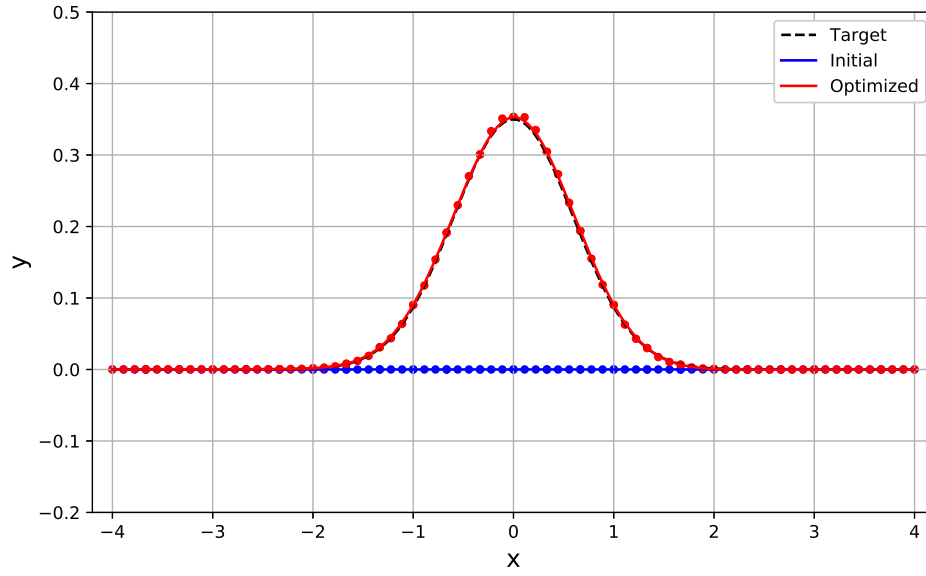
(a) Target Mesh (96 x 16)



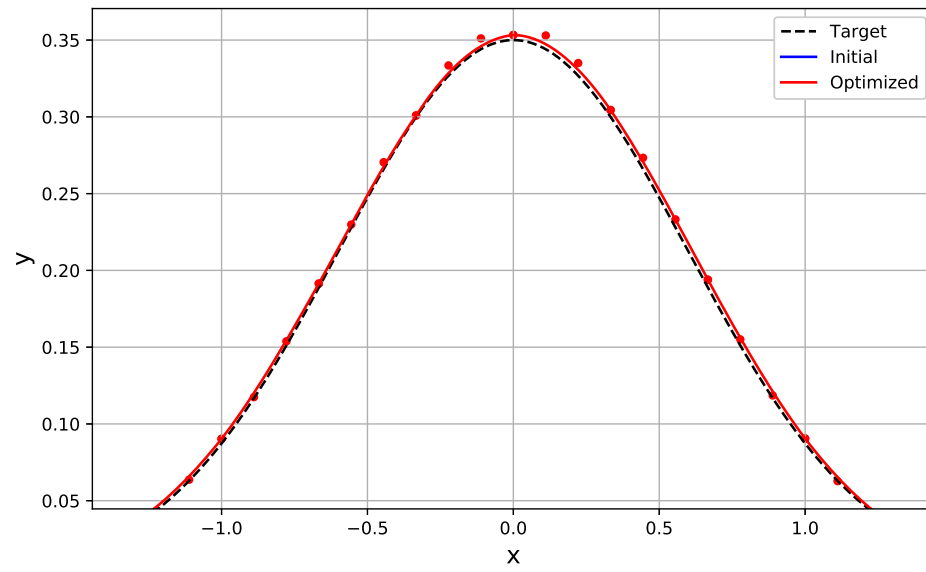
(b) Initial Mesh (24 x 4)

Figure 5–1: The initial and target mesh used for the optimization.

the coarseness of the design mesh used. The pressure distribution is also shown in Fig. (5–3), where the pressure over the optimized and target channel face has been plotted normalized with respect to the total pressure. This pressure distribution has been plotted as a function of the arc length position normalized by the total length of the channel face. As Fig. (5–3) shows, the optimized profile has a distribution that is quite close to the target.



(a) Final optimized bump profile



(b) Zoomed in image of optimized bump profile

Figure 5–2: The initial, optimized and target profiles for the inviscid channel bump optimization case. The points shown are the design control points.

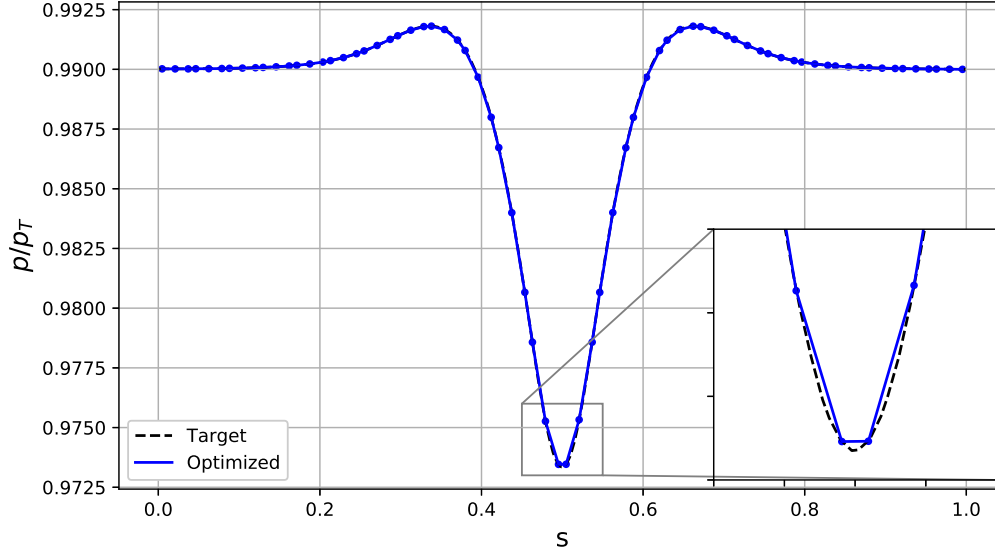


Figure 5–3: The optimized and target pressure distribution over the bump normalized by the total pressure. The x axis displays the arc length position along the channel normalized by its total arc length.

The convergence of the cost function and the L2 norm of its gradient with respect to the design variables is also shown in Fig. (5–4). It can be seen that the total optimization process required 550 design iterations for the gradient of the cost function to drop to a value below $1e - 9$. The slower convergence for the first 100 iterations is primarily a result of the explicit smoothing operator incorporated into the optimization. The number of smoothing iterations needed at each design iteration is determined as a function of the L2 norm of the search vector $\mathbf{p}_k$, where a larger L2 norm results in more iterations to be applied. Moreover, each smoothing iteration reduces the magnitude of the search direction, resulting in a smaller overall

change in the shape. Since in the first design iteration the search vector has a larger norm, more smoothing iterations are required which in turn slows the convergence.

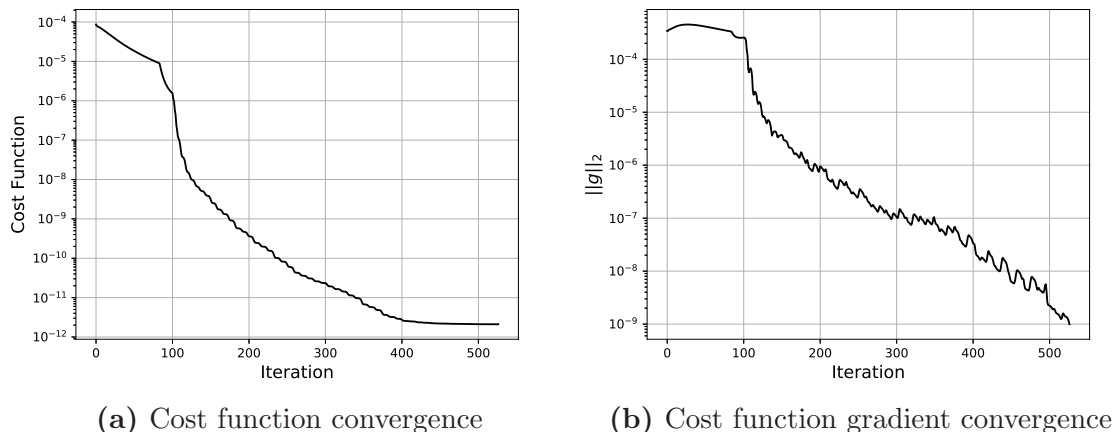


Figure 5–4: The convergence of the cost function and L2 norm of its gradient with respect to the design iteration for the inviscid channel case.

5.2 NACA 0012 Inverse Design

The next case that is presented is the inverse design of a NACA 0012 airfoil. In this case, a NACA 4412 airfoil is considered as the target profile, while the NACA 0012 airfoil is treated as the initial profile. Shown in Fig. (5–5) is a plot of these two airfoils. Moreover, the cost function used in the optimization is identical to that used in the inviscid channel inverse design case, and it is given in equation (5.1).

For this optimization case, an angle of attack of $\alpha = 0^\circ$ and freestream Mach number of $M = 0.1$ is used. These parameters have been applied through a set of Reimann boundary conditions on the mesh boundary, while a slip wall boundary condition is applied on the airfoil surface. Moreover, for this high-order case, the order of the basis functions is $P = 2$.

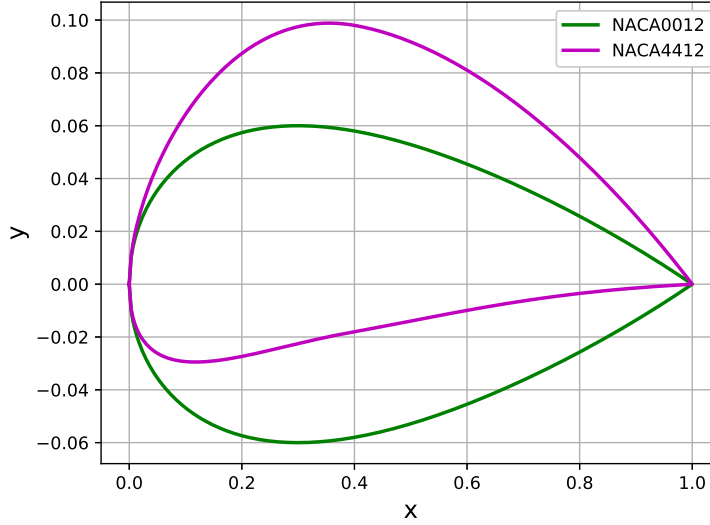
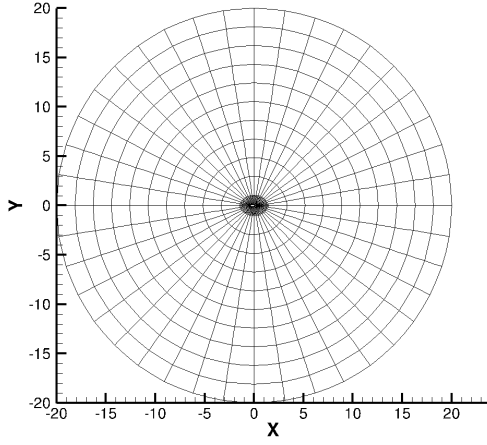


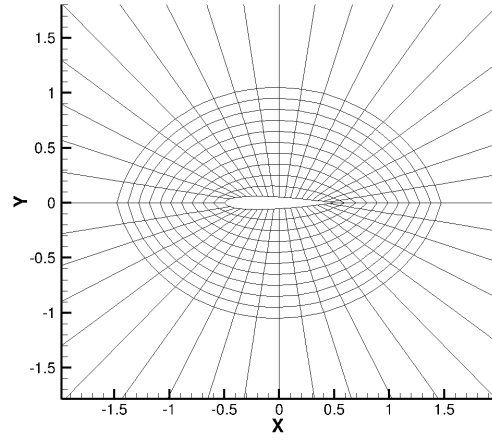
Figure 5–5: A plot of the NACA0012 and NACA4412 airfoil profiles.

The mesh around both the target airfoil and initial airfoil are O-grid type meshes, and each are of dimension 40×20 . A sample plot of this mesh is shown for the NACA 0012 airfoil in Fig. (5–6). Furthermore, the dimension used for this mesh results in 78 design variables that can freely move. In this case, once again, the displacement of these design variables is restricted to only the y -direction. Moreover, the design variables at the leading and trailing edge of the profile are kept fixed.

Fig. (5–7) shows the initial, optimized and target profiles of the airfoils used in the optimization. As the close up plots in the figure show, the high-order optimization scheme is able to quite accurately capture the trailing edge of the target profile, while a difference in these airfoils can be seen near the lower surface at the leading edge. This is primarily due to the fact that a maximum of 350 design iterations are used for this case due to the high computational cost of each iteration. This results in



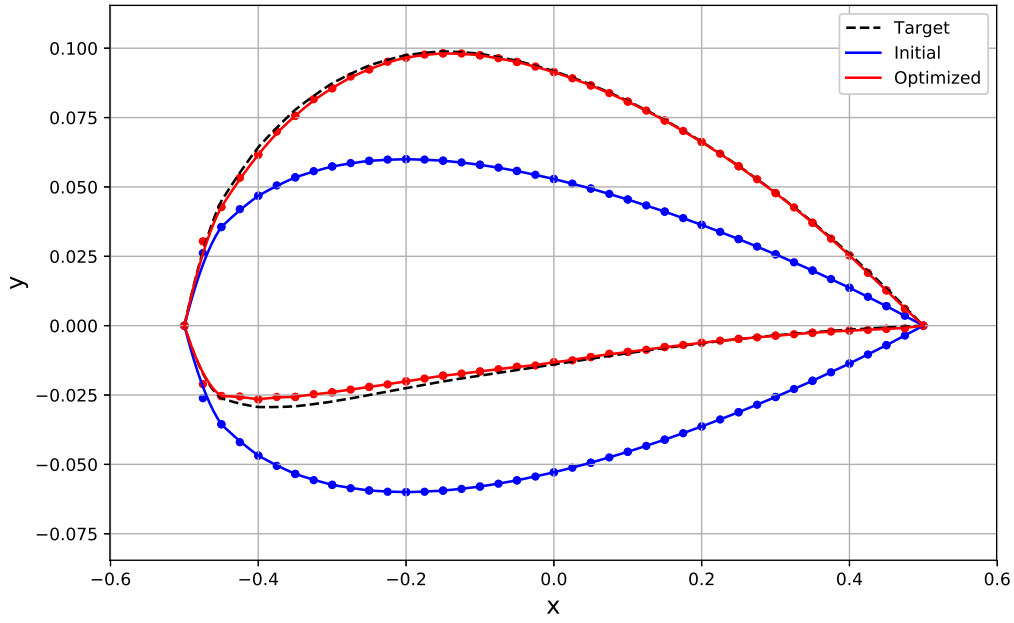
(a) O-grid used for the design mesh



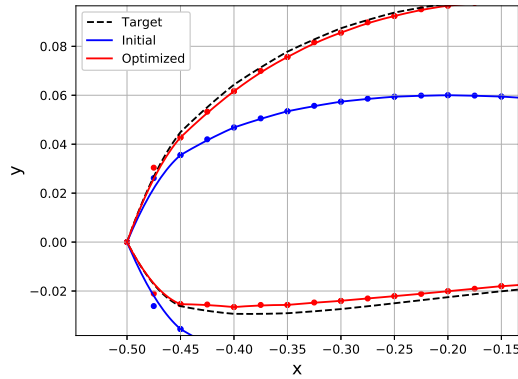
(b) Zoomed in view of the O-grid

Figure 5–6: The mesh placed around the NACA 0012 design airfoil that was optimized.

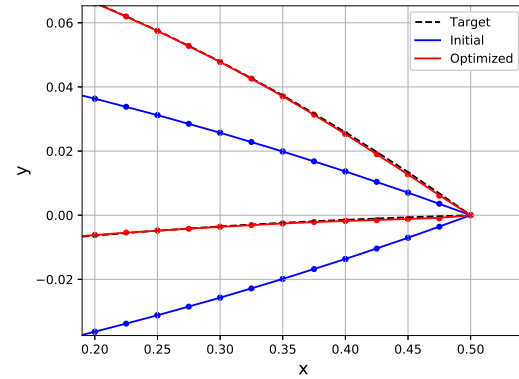
the L2 norm of the gradient attaining a minimum value on the order of 10^{-6} , which in turn means that a further reduction in the cost function is possible. Convergence plots of the cost function and its gradient are shown in Fig. (5–8).



(a) Initial, optimized and target profiles. The points shown are the design control points.



(b) Close up view of the leading edge



(c) Close up view of the trailing edge

Figure 5–7: The initial, optimized and target profiles. The mesh dimension is 40×20 and $P = 2$.

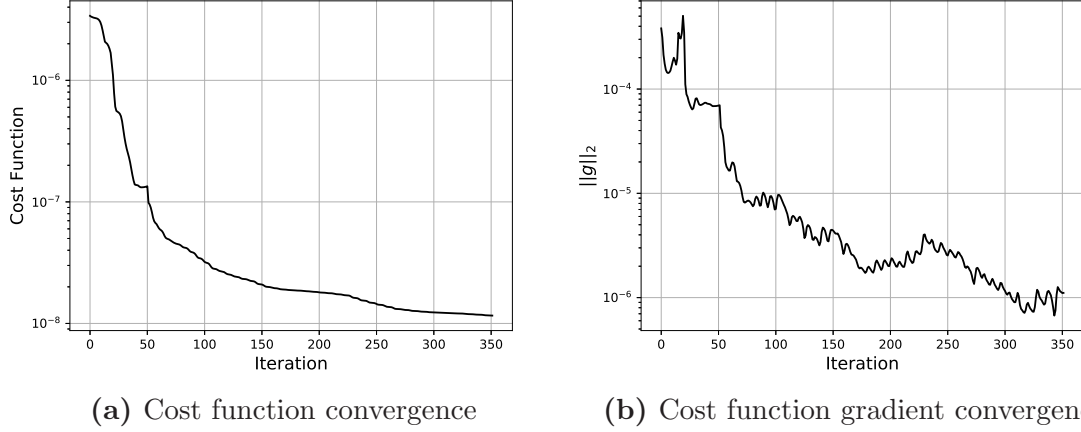


Figure 5–8: The convergence of the cost function and L2 norm of its gradient with respect to the design iteration for the NACA 0012 inverse design case.

5.3 NACA 0012 Target Lift Coefficient

The final case that is presented is a target lift coefficient optimization case. In this optimization problem, the NACA 0012 airfoil is used as the initial profile, which has a C_L of 0. A fixed target lift coefficient value of $C_{L, target} = 0.24$ is then specified, and the goal in the optimization is to be able to attain this specified lift. The cost function for the optimization problem in this case is given by

$$I = (C_L - C_{L, Target})^2. \quad (5.2)$$

The target lift coefficient case provides a perfect means to test the true effectiveness of using high-order methods for optimization. This is done by evaluating this airfoil case for several different P values while keeping the number of design variables and total flow variables approximately constant, so that the computational cost for each case is approximately equivalent. The P values used for this case have been set

Table 5–1: The properties of the meshes used for the target lift coefficient optimization case.

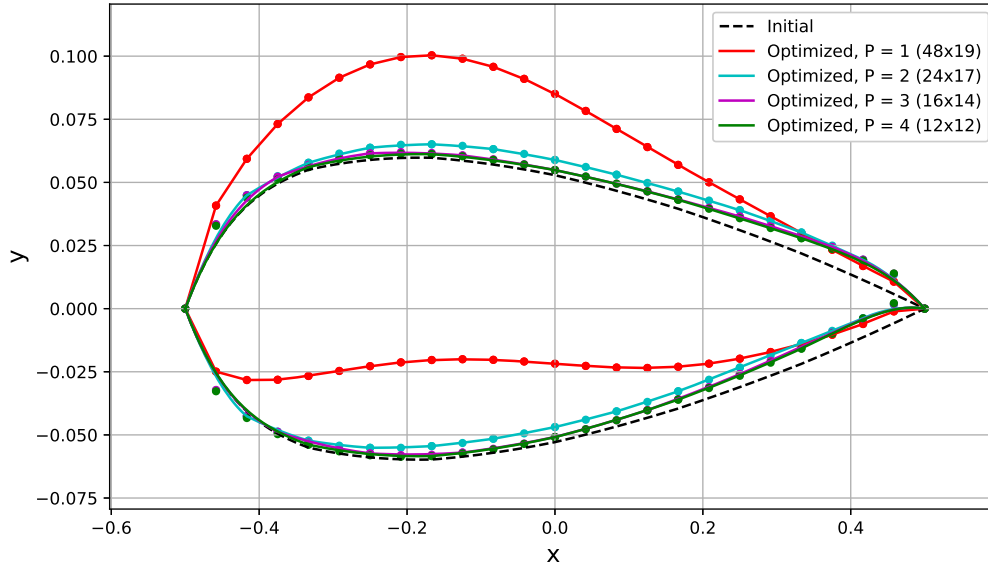
Polynomial	Mesh Dimension	Number of Design Variables	Number of Flow Variables
1	48 x 19	46	3648
2	24 x 17	46	3672
3	16 x 14	46	3584
4	12 x 12	46	3600

to be $P = 1, 2, 3$ and 4, and the mesh dimensions used to ensure an equal number of variables are shown in Table (5–1).

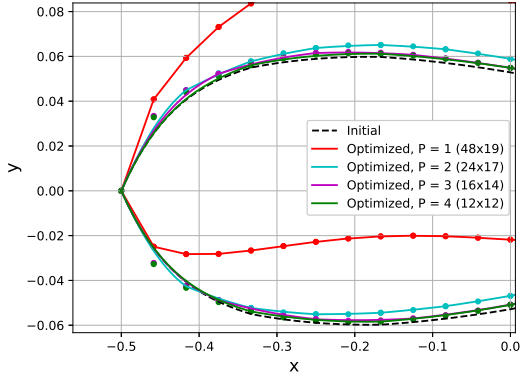
For all cases, the boundary conditions used were identical to those used in the airfoil inverse design optimization case. Moreover, all cases have been run until the L2 norm of the gradient of the cost function decreased to a value below $1e - 9$.

Fig. (5–9) depicts the optimized results for all cases tested. As can be seen, the $P = 1$ case converges to a very different solution to the optimization problem. The $P = 2, 3$ and 4 cases, however, all converge to approximately the same optimized airfoil shape in which the trailing edge is adjusted slightly to attain the specified lift. The convergence plots also reveal an interesting trend in Fig. (5–10), Fig. (5–11) and Fig. (5–12). Firstly, it can be seen that the $P = 1$ case takes much longer to converge. This however is a result of the fact that this case converges to a different optimum solution. In order to make an effective comparison using the $P = 1$ case, an additional pitching moment constraint will be included in the optimization and investigated in a future work, as this will ensure all airfoils converge to the same optimum solution.

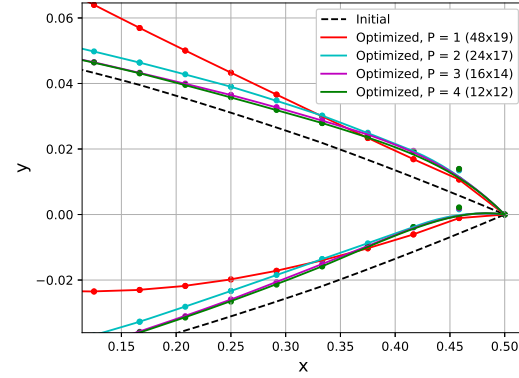
Regardless of the $P = 1$ convergence trend, a comparison can still be made between the $P = 2, 3$ and 4 cases. Firstly, all plots for CPU time show that approximately the same amount of time is needed for the convergence, an expected result since the number of flow design variables was kept approximately constant. Moreover, in general, it can be seen that the higher values of P results in fewer design iterations. This may be attributed firstly to the increased smoothness that results from representing the airfoil with higher order polynomials. The higher P cases are able to parametrize the airfoil shape using fewer elements. For the airfoil surface, only C^0 continuity is ensured between elements, whereas the continuity over a given element boundary is C^∞ . Due to the lower number of elements needed for the higher P cases, better continuity is afforded when parametrizing the airfoil shape which allows for a smoother surface representation. In shape optimization cases, smoother geometries result in the ability to use larger step lengths in the optimization. This, in turn, may be a first explanation of the quicker convergence for the higher P cases. An additional cause for the accelerated convergence is also the increased accuracy that yields from using high values for P . Higher order polynomials provide a more accurate numerical solution which translates to a more accurate representation of the cost function. The increased accuracy of this cost function should result in more accurate gradients which improves the ability to evaluate the search direction for the optimization. Further studies must be completed to verify this fact. The results shown, however, are promising, as they demonstrate a clear advantage for using high-order methods in optimization applications.



(a) Initial and optimized airfoil profiles

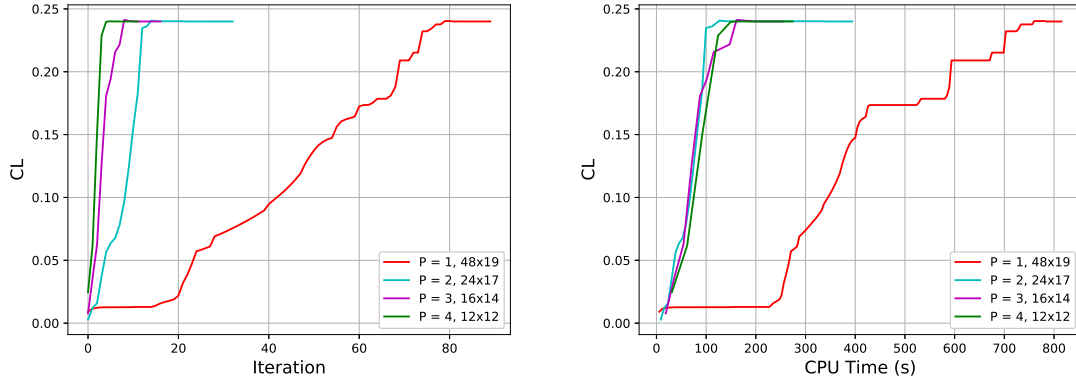


(b) Close up view of the leading edge



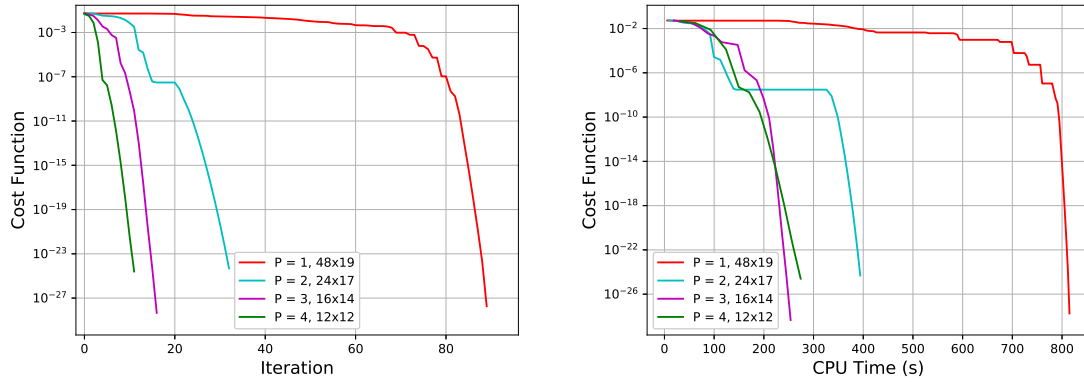
(c) Close up view of the trailing edge

Figure 5–9: Initial and optimized airfoil profiles for the target lift coefficient optimization case. Shown in brackets in the legend is the dimension the mesh used for the given P case.



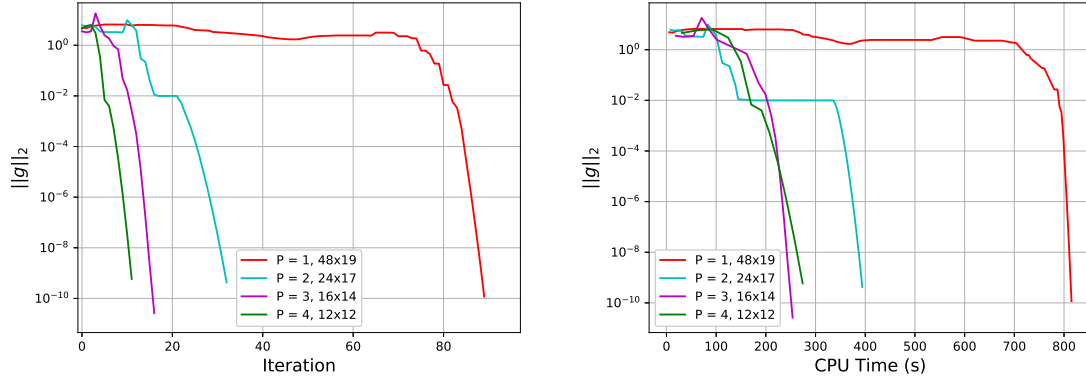
(a) Convergence as a function of iterations (b) Convergence as a function CPU time

Figure 5-10: Convergence of the lift coefficient as a function of design iterations and CPU time.



(a) Convergence as a function of iterations (b) Convergence as a function of CPU time

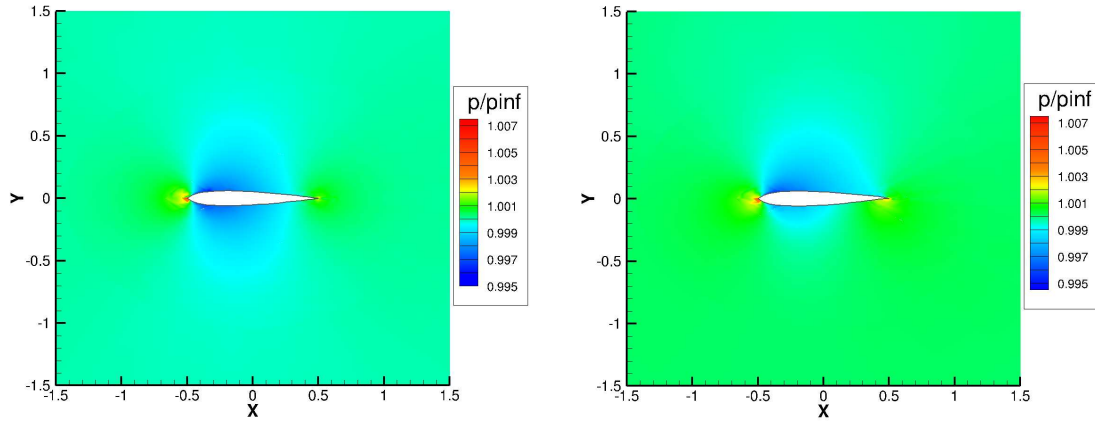
Figure 5-11: Convergence of the cost function as a function of design iterations and CPU time for the target lift coefficient case.



(a) Convergence as a function of iterations (b) Convergence as a function of CPU time

Figure 5-12: Convergence of the gradient of the cost function as a function of design iterations and CPU time for the target lift coefficient case.

Shown in Fig. (5-13) is the pressure distribution around the optimized $P = 4$ airfoil. The higher pressure on the lower face of the optimized airfoil shows clearly how the lift is generated to attain the target lift coefficient.



(a) Initial pressure distribution

(b) Optimized pressure distribution

Figure 5-13: The pressure distribution, normalized by the freestream pressure, over the initial and optimized airfoil for the $P = 4$ case with a mesh dimension of 12×12 .

CHAPTER 6

Conclusion

This work has been motivated by the need to investigate the efficacy of high-order methods when paired with adjoint based optimization. The novel contributions of this study consists of the implementation of an adjoint solver with a high-order method, namely the Discontinuous Galerkin method.

The effectiveness of the high-order approach to optimization is shown in this work by studying several cases. First, successful results to inverse design problems are presented. An inverse design problem consisting of internal flow through a channel is first shown. This is followed by results showing the inverse design of a NACA 0012 airfoil. The correct optimal results obtained for these cases shows that high-order methods can indeed be used with the adjoint approach in optimization settings.

An optimization case is also presented in which a target lift coefficient is specified and the given airfoil must be optimized to attain this value. In this case, the effectiveness of higher orders of accuracy is measured, by comparing both the number of design iterations as well as the CPU time needed to obtain the optimum solution. The preliminary results show that increased orders of accuracy are indeed effective for optimization, as they result in a decrease in the net number of design iterations, while having a negligible effect on the CPU time needed. If this trend continues to more complicated problems, higher order optimization schemes may prove to be a very effective approach for optimization in the future.

6.1 Future Work

In CFD, one very important challenge that arises is the ability to represent the geometry around which the flow is being solved. The accurate representation of the geometry is a fundamental requirement to obtain meaningful solutions. Current high-order methods through the use of curved elements try to address this issue, as they allow for the use of polynomials of high degrees to approximate the body around which the flow is computed. Although this has produced very favorable results, a relatively novel concept known as Isogeometric Analysis (IGA) offers an alternate approach to alleviate issues with geometrical representations.

Introduced by Hughes, Cottrell and Bazilevs [5], IGA consist of using Non-Uniform Rational B-Spline (NURBS) basis functions, the same functions used in CAD programs to represent the geometry, to form the function space to represent the flow’s numerical solution. This in turn results in no approximation needing to be made to represent the geometry in the analysis (the geometry can be represented exactly when solving the flow).

The incorporation of IGA into a high-order optimization code should prove to be a very powerful optimization algorithm. The ability to exactly represent geometrical CAD models in the analysis will allow for this approach to be able to directly read CAD files, optimize the exact geoemtry as specified by the user, and return this geometry in an updated CAD file. Current approaches require approximations to the geometry to be made when transforming from the CAD model to the generate the geometric model used in the analysis and optimization, a step that is completely eliminated using IGA. Ultimately, if the results prove to be promising, optimization

schemes using adjoint-based optimization paired with high-order methods incorporating IGA should prove to have a significant advantage over current methodologies in place.

REFERENCES

- [1] J.S. Hesthaven and T. Warburton. *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*. Springer, 2008.
- [2] F. Bassi, S. Rebay. High-order accurate discontinuous finite element solution of the 2D Euler equations. *Journal of Computational Physics*, 138(2):251-285, 1997.
- [3] M.J. Borden, M.A. Scott, J.A. Evans, T.J.R. Hughes. Isogeometric finite element data structures based on Bézier extraction of NURBS. *International Journal for Numerical Methods in Engineering*, 87(15):15-47, 2011
- [4] B. A. Szabo and I. Babuka, *Introduction to finite element analysis: formulation, verification, and validation*. John Wiley and Sons, 1991.
- [5] T.J.R. Hughes, J.A. Cottrell, and Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39-41):4135-4195, 2005.
- [6] J. A. Cottrell, Y. Bazilevs, and T. J. R. Hughes, *Isogeometric analysis: toward integration of CAD and FEA*. Chichester: Wiley, 2009.
- [7] Witherden, F. D. and Jameson, A.: Future Directions in Computational Fluid Dynamics. AIAA 2017-3791, doi: 10.2514/6.2017-3791, 35th AIAA Applied Aerodynamics Conference, AIAA AVIATION Forum, 5-9 June 2017, Denver, CO.
- [8] F. Bisson and S. Nadarajah, "Adjoint-Based Aerodynamic Design Optimization Using the Drag Decomposition Method," *31st AIAA Applied Aerodynamics Conference*, 2013.
- [9] Giles, M. B. and Pierce, N. A., "An introduction to the adjoint approach to design," *Flow, Turbulence and Combustion*, vol. 65, no. 34, pp. 393-415, 2000.
- [10] Nadarajah, S., "A comparison of the discrete and continuous adjoint approach to automatic aerodynamic optimization," in *Proceedings of the 38th Aerospace*

Sciences Meeting and Exhibit, American Institute of Aeronautics and Astronautics, Reno, Nev, USA, 2000, AIAA paper 2000-0667.

- [11] E. F. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics A Practical Introduction*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.
- [12] P. Zwanenburg and S. Nadarajah, "Equivalence between the Energy Stable Flux Reconstruction and Filtered Discontinuous Galerkin Schemes: Numerical Validation," 54th AIAA Aerospace Sciences Meeting, Feb. 2016.
- [13] L. A. Piegl and W. Tiller, *The NURBS book*. Berlin: Springer, 1997.
- [14] V. Poirier and S. Nadarajah, "Efficient Reduced-Radial Basis Function-Based Mesh Deformation Within an Adjoint-Based Aerodynamic Optimization Framework," *Journal of Aircraft*, vol. 53, no. 6, pp. 1905-1921, 2016.
- [15] Z. Ji, L. Liu, and G. Wang, "A Global Laplacian Smoothing Approach with Feature Preservation," *Ninth International Conference on Computer Aided Design and Computer Graphics (CAD-CG05)*.
- [16] A. Jameson. Aerodynamic design via control theory. *Journal of Scientific Computing*, 3(3):233-260, 1988.
- [17] S. Nadarajah, P. Castonguay, and A. Mousavi, "Survey of Shape Parameterization Techniques and its Effect on Three-Dimensional Aerodynamic Shape Optimization," *18th AIAA Computational Fluid Dynamics Conference*, 2007.
- [18] J. Nocedal and S. Wright, *Numerical Optimization*. New York: Springer-Verlag, 2006.